

```
$ copilot --structured --mode agent --duration 60min
```

From Vibe Coding to Structured AI Development

GitHub Copilot in VS Code — 1-Hour Workshop



60 min | 7 demos | 1 live build

Agenda

60 minutes — paced with live demos throughout

0-5 min

01

The Landscape

Copilot modes, what changed with agents

5-12 min

02

Vibe vs. Structured

What goes wrong and why

12-22 min

03

Live Exercise — Part 1

Build a CLI tool: vague prompt path

22-32 min

04

Adding Structure

AGENTS.md, copilot-instructions.md, llm.txt

32-42 min

05

Live Exercise — Part 2

Same task, structured path

42-50 min

06

Tools, Skills & MCP

The agent's full toolkit + ascii-art skill demo

50-56 min

07

Agents

Out-of-the-box & custom agents

56-60 min

08

What's Next + Q&A

RPIV, spec-driven dev, harnesses

01 The Landscape

Copilot modes, what changed with agents

GitHub Copilot in VS Code

Four interaction modes — each with different expectations



Inline Completions

Tab to accept as you type.
Single-line or small block. Zero
context-setting needed.



Chat

Ask questions, explain code,
get snippets. You drive; Copilot
responds.



Edits (multi-file)

Describe a change; Copilot
edits across files. Needs clear
scope.



Agent Mode

Copilot plans, runs tools,
iterates autonomously. Needs
structure to stay on track.

This session focuses on Agent Mode — where structure matters most.

02 Vibe vs. Structured

What goes wrong — and why

Why Vibe Coding Fails

The agent can only work with what you give it



Vague Prompt

"make a CLI that does ASCII art"



Agent Guesses

Picks a library, style, framework



Wrong Direction

Not what you meant — but it's committed



Back-and-Forth

4–6 correction cycles to recover

No goal

Agent doesn't know success criteria

No constraints

Any approach is valid — agent picks arbitrarily

No examples

Agent has no reference point for style or output

03 Live Exercise — Part 1

Build a CLI tool: the vague prompt path

What We're Building

A small, focused tool — enough to show the difference clearly



```
$ ascii-art --name "Juan" --style fire --animate
```



CLI Interface

Takes --name, --style, --animate flags



ASCII Output

Renders the name as animated ASCII art



Node.js / Python

Student's choice of runtime



No external APIs

Self-contained, no network calls needed

Demo: Open VS Code → Agent mode → paste the vague prompt. Watch what happens.

The Vague Prompt Path

What the agent does when you leave it to guess

You said

"create a CLI that does ASCII art"

"make it animated"

"use something popular"

Agent assumed

- Library: figlet + chalk
(maybe not what you wanted)
- Style: random default
(no --style flag implemented)
- Animation: setInterval
(flickers on Windows terminals)
- Language: Node.js
(you might have meant Python)

Result: working code — but not YOUR vision. Now you spend time correcting, not building.

04 Adding Structure

AGENTS.md, copilot-instructions.md, llm.txt

The Agent's Briefing Documents

Three files that give the agent context before it starts



AGENTS.md

Project root

Agent config: goal, scope, allowed tools, constraints

```
Goal: build a Node.js CLI
Runtime: Node 18+, no ESM
Do NOT use external APIs
Output format: ANSI escape codes
```

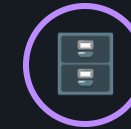


copilot-instructions.md

.github/ folder

Copilot-specific: coding style, stack, dos & don'ts

```
Use TypeScript strict mode
Prefer functional patterns
Test with Vitest, not Jest
No default exports
```



llm.txt

Project root

Any LLM: concise facts, examples, scope limits

```
# ascii-art CLI
Inputs: --name, --style, --animate
Outputs: terminal-rendered ASCII
Styles: fire, matrix, ghost
```

05 Live Exercise — Part 2

Same task — now with structure in place

Before & After — Same Task, Different Setup

What the agent produces when it has context

WITHOUT instructions

- Picked figlet + chalk (wrong version)
- Ignored --style flag entirely
- Used require() + ESM mix — broke on run
- No error handling for missing --name
- Animation flickers on Windows
- 6 follow-up prompts to fix

WITH instructions

- ✓ Used correct library (pure Node, no deps)
- ✓ All three flags implemented correctly
- ✓ CJS throughout — runs on first attempt
- ✓ Graceful error for missing --name
- ✓ ANSI frames — works cross-platform
- ✓ Zero follow-up corrections needed

06 Tools, Skills & MCP

The agent's full toolkit

What the Agent Can Do

Tools are the agent's hands — MCP extends its reach



Tools (built-in)

Read / Write files

Edit source without copy-paste

Run terminal commands

Build, test, install deps

Search codebase

Find patterns across files

Open browser

Fetch docs, validate URLs



MCP (external)

Always-on services

GitHub, Jira, Confluence, Azure

Retrieve live data

PRs, issues, wiki pages, logs

Push changes

Create PRs, update tickets

Custom connectors

Your own internal APIs

Skills — Knowledge Packs for Agents

Filesystem-based context the agent loads when needed



What is a Skill?

- A folder with a SKILL.md (instructions) and any reference files
- Agent reads it when the task matches the skill's trigger
- Encapsulates domain knowledge, patterns, constraints

```
.github/skills/  
  ascii-art/  
    SKILL.md  
  examples/  
    fire-example.txt  
    matrix-example.txt
```

Demo: Create `ascii-art/SKILL.md` → use it in Agent mode → agent reads it before generating code

Model & Config — Quick Take

Model: speed vs. depth | Context: bigger = more code visible | Effort: high = deliberate output

07 Agents

Out-of-the-box & custom — when and which

Out-of-the-Box vs. Custom Agents

Pick the right agent for the job



Out-of-the-Box

Research agent

Multi-source web search + plan

Code review agent

Static analysis + suggestions

PR summary agent

Diff → readable description

Workspace agent

Index + query your whole codebase



Custom Agents

Licensed sources

Exa, Perplexity, MS Docs via MCP

Custom tool chain

Internal APIs, private registries

Domain-specific

Knows YOUR codebase conventions

Composable

Chain multiple agents together

Demo: Research agent → "Find CLI tools for ASCII diagrams and art" → plan → search → validate → code

08 What's Next

RPIV, spec-driven dev, harnesses & Q&A

Key Takeaways & What Comes Next

Today's takeaways

- ✓ Agent mode needs structure — not just good prompts
- ✓ AGENTS.md, copilot-instructions.md, and llm.txt are your briefing docs
- ✓ Tools + MCP + Skills give the agent reach without hallucination
- ✓ Model & config choices change behaviour, not just speed
- ✓ Instructions = reproducible, reviewable, version-controlled

Today

Structured AI Dev

Next

RPIV Framework

Then

Spec-Driven Dev

Future

Harnesses & Loops

Questions?

Let's build something together.

Repo

`github.com/ise-asia-au/copilot-workshop`

Docs

`docs.github.com/copilot`

Contact

`juan.burckhardt@microsoft.com`